

Proxmox Storage DRS -- Internals

How the tool works, for whoever changes it

Bernd Zeimetz bernd@bzed.de

2026-09-05

Rendered from docs/internals/*.md, SHA-256 31aefab07eac0f91d6639458c9ab3e96cb39464f01196ecef517b7045f0292ad
(sha256sum --check docs/internals.pdf.sha256 in the repository must succeed).

Copyright © 2026 Bernd Zeimetz. Licensed under the GNU Affero General Public License, version 3 or later.

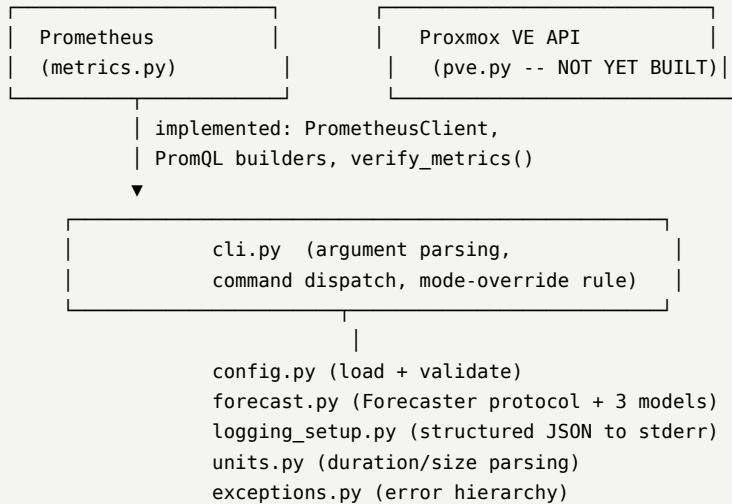
Contents

The pipeline, as specified and as built	3
Module layout, as it stands	3
Why config.py depends on forecast.py	4
Where to read next	4
How a YAML file becomes a validated Config	4
Resolution order	4
The two-stage validation	4
Secrets from the environment	5
What is deliberately <i>not</i> checked here	5
ResolvedConfig: what the caller actually gets	5
Forecasting: the protocol and its three implementations	5
The one rule, in one place	5
Why the upper bound, never the point estimate	5
QuantileForecaster	5
SeasonalNaiveForecaster	6
HoltWintersForecaster	6
build_forecaster()	6
The Prometheus client and verify-metrics	6
_SessionLike / _ResponseLike: why not requests.Session directly	6
_get(): one request path, three endpoint shapes	7
PromQL construction	7
verify_metrics(): six checks, six functions	7
What cli.py does with the report	7
Command dispatch, the mode-override rule, and why logs go to stderr	7
One parser, global options first	8
Command handlers: a dict, not a chain of ifs	8
The --mode escalation rule	8
Why logs go to stderr, not stdout	8
JsonFormatter: what ends up in one log line	8
--manual: preferring man(1), falling back only when necessary	9

What does this page answer? Where does an invocation of pve-storage-drs go, which module owns which piece, and how much of IMPLEMENTATION_PLAN.md's section 2 architecture actually exists right now?

The pipeline, as specified and as built

IMPLEMENTATION_PLAN.md section 2 describes seven stages: collect, join, gate, solve, cost, order, execute. As of this page, stages 1 (partially — Prometheus only, no PVE API yet) and the surrounding configuration/CLI scaffolding exist; join through execute are IMPLEMENTATION_PLAN.md section 12 phases 2-9 and are not yet written. Do not take this page as a claim that the whole pipeline runs end to end — [../manual/30-safety-and-status.md](#) is the authoritative per-command status.



Module layout, as it stands

Module	Responsibility	Plan section
exceptions.py	The DrsError hierarchy every deliberate failure raises	AGENTS.md section 5
units.py	Duration/size parsing ("24h", "200MiB") and human-readable formatting	section 11
config.py	Load /etc/pve/drs.yaml, jsonschema + semantic validation, the frozen dataclass config model	section 11
config_schema.json	The jsonschema structural half of validation	section 11.1
forecast.py	The Forecaster protocol, required_range_seconds, and quantile/seasonal_naive/holt_winters	section 10
logging_setup.py	Structured JSON logging to stderr	section 2.1 (amended, see 40-cli-and-logging.md)
metrics.py	PrometheusClient, PromQL construction, verify_metrics()	sections 3.1-3.4
cli.py	Argument parsing, command dispatch, --manual, the mode-override rule	section 11.3

Not yet written: pve.py, topology.py, loadmodel.py, optimize.py, heuristic.py, payback.py, schedule.py, execute.py.

Why config.py depends on forecast.py

config.py's semantic validation (`_check_forecast_window`) must reject a `window.lookback` too short for the configured `forecast.model` at startup (section 11.1) — that check needs `forecast.required_range_seconds()`. Since `forecast.py` in turn needs `config.ForecastConfig` for its type signatures, the import is deferred to inside the validation function rather than at module level, breaking what would otherwise be an import cycle. See the comment at the top of `config.py`'s `_check_forecast_window` if you touch this.

Where to read next

- [10-configuration.md](#) — how a YAML file becomes a validated `Config`, and where every default actually lives.
- [20-forecasting.md](#) — the forecaster protocol and its three implementations.
- [30-metrics.md](#) — the Prometheus client and the six `verify-metrics` checks.
- [40-cli-and-logging.md](#) — command dispatch, the `--mode` escalation rule, and why logs go to `stderr`.

How a YAML file becomes a validated Config

What does this page answer? What exactly happens between `pve-storage-drs -c foo.yaml plan` and a type-checked `Config` object, and where does each of the section 11.1 validation rules actually live? Describes `proxmox_storage_drs/config.py` and `config_schema.json`.

Resolution order

`resolve_config_path()` implements `IMPLEMENTATION_PLAN.md` section 11's three-source order: `--config` (`cli_path`), then `$PVE_STORAGE_DRS_CONFIG`, then `config.DEFAULT_CONFIG_PATH` (`/etc/pve/drs.yaml`). It also returns `was_explicit`, which `load_config()` uses to decide the error message on a read failure: an explicitly named path that cannot be read is always fatal with no fallback, per section 11, but the *message* additionally points at `config/drs.example.yaml` only when the failing path was the unnamed default — there is nothing useful to suggest when the operator named the path themselves.

The two-stage validation

1. `_validate_schema()` loads `config_schema.json` via `importlib.resources` (it ships as package data — see `pyproject.toml`'s `[tool.setuptools.package-data]`) and runs it through `jsonschema`. Every object in the schema sets `additionalProperties: false`; a typo'd key is caught here as a validation error, not silently ignored, which is what keeps section 15.1's knob-to-formula table trustworthy (a key with no formula would otherwise go unnoticed).
2. `_build_config()` walks the now-schema-valid raw dict into the frozen dataclasses defined earlier in the module, applying every default inline via `.get(key, default)`. These defaults are also what `config/drs.example.yaml` documents and what the manual's [../manual/10-configuration.md](#) describes — there is deliberately no second copy of a default anywhere else in the codebase.
3. `_validate_semantics()` runs the section 11.1 rules `jsonschema` cannot express — cross-field comparisons, or rules needing a value derived from two independently-optional settings. Each rule is a small `_check_*` function appending to a shared `errors/warnings` list (kept separate functions rather than one large one, partly for flake8's max-complexity and partly because each is independently testable). errors become one `ConfigError` naming every problem found, not just the first; warnings are returned to the caller (`ResolvedConfig.warnings`) for `cli.py` to log — section 11.1's `saturation_load` check is the one warning-only rule today.

Every duration or size field is parsed **once**, at this point, via `units.parse_duration_seconds/parse_size_bytes`, and stored on the dataclass already in its canonical unit (seconds, bytes) — nothing downstream re-parses a string.

Secrets from the environment

`_build_config()` fills `proxmox.auth.password/token_secret` from `PVE_PASSWORD/PVE_TOKEN_SECRET` **only when the file's own value is null** — a secret actually written in the file is used as-is, never silently overridden by an environment variable a deploy script forgot to unset. This one-directional fallback is what section 11's "keep secrets out of `/etc/pve`" guidance is for: the file can omit the field entirely and rely on the environment, or set it explicitly and take responsibility for the file's own permissions.

What is deliberately *not* checked here

Two section 11.1 rules need live cluster data this module does not have and are therefore **not** part of `config.py`:

- storage ids actually existing in the cluster;
- `execution.source_release.timeout` / `gates.cooldown_per_storage` against a storage's real `saferemove_throughput`.

Both belong to `pve.py/topology.py` (not yet written) and `pve-storage-drs verify-storages`, once they exist — see [../manual/30-safety-and-status.md](#) for current status.

ResolvedConfig: what the caller actually gets

`load_config()` returns a `ResolvedConfig`, not a bare `Config`: the resolved path and the file's sha256 travel with it, because `IMPLEMENTATION_PLAN.md` section 11 requires every run to log both — a plan must always be traceable to the exact file that produced it, even though the file is never re-read after startup.

Forecasting: the protocol and its three implementations

What does this page answer? How does `forecast.py` decide how much history a model needs, and what does each of `quantile/seasonal_naive/holt_winters` actually compute? Describes `proxmox_storage_drs/forecast.py`.

The one rule, in one place

`IMPLEMENTATION_PLAN.md` section 10.1's table says how much history each model needs, independent of `window.lookback` (the *decision* window). That rule has exactly one implementation: three small pure functions, `_quantile_required_range_seconds`, `_seasonal_naive_required_range_seconds` and `_holt_winters_required_range_seconds`, each called from **two** places — the free function `required_range_seconds()` that `config.py`'s startup validation uses (before any `Forecaster` is constructed), and the matching `Forecaster.required_range()` method on the concrete class. Duplicating the arithmetic between those two call sites, rather than sharing the pure function, is exactly the kind of "second copy of a rule" `AGENTS.md` section 5 forbids — a change to the Holt-Winters requirement made in only one of them would silently desynchronize config validation from what the forecaster itself believes it needs.

Why the upper bound, never the point estimate

Every `Forecaster.predict()` returns a `Forecast(point_estimate, upper_bound)`. Nothing in this module enforces that callers use `upper_bound` — that discipline belongs to the caller (the optimizer and the section 7.3 saturation guard, neither written yet). The asymmetry is stated in the module docstring and repeated here because it is easy to get backwards under time pressure: overestimating costs a slightly worse balance, underestimating risks scheduling a mirror onto a storage that is about to saturate.

QuantileForecaster

The default. `predict()` takes the `window.quantile/window.upper_quantile` percentiles of the raw series with no fitting at all. `_quantile()` implements linear-interpolation percentile matching `numpy.percentile`'s default method, by hand — deliberately not using `numpy`, since the tool must stay

usable with only `requests + ruamel.yaml + jsonschema` installed (IMPLEMENTATION_PLAN.md section 2.1). horizon is accepted (to satisfy the Forecaster protocol) and immediately discarded: the quantile model has no notion of a horizon-dependent forecast.

SeasonalNaiveForecaster

Groups historical samples by hour-of-day (`int((ts // 3600) % 24)`) and takes the median as the point estimate, the configured upper quantile across the same bucket as the bound. `now_epoch_seconds` selects which hour-of-day bucket the *current* moment falls into; `predict()` ignores everything outside that bucket. A bucket with no samples returns `Forecast(0.0, 0.0)` rather than raising, matching IMPLEMENTATION_PLAN.md section 4's rule for an idle group's raw quantities: no data is zero, not an error, at this layer (the caller decides whether zero is trustworthy).

HoltWintersForecaster

`statsmodels` is an optional dependency (Suggests, not Depends — see `debian/control`), so it is imported **only inside `predict()`**, never at module level; `debian/tests/import-all` is what would catch a regression here. Two situations fall back to `QuantileForecaster`, both logged at warning level rather than failing silently:

1. fewer than `2 * seasonal_periods` samples in the series (section 10.2's own rule — a fit on too little data is worse than no fit);
2. `statsmodels` is not importable at all.

The upper bound is `point_estimate + residual_z * stdev(in-sample residuals)` — not derived from the model's own confidence interval, which `statsmodels's ExponentialSmoothing.fit()` does not expose directly for every configuration. `residual_z` (`forecast.holt_winters.residual_z`, default 2.0) is what an operator tunes if this bound turns out too tight or too loose in practice; there is no plan-mandated backtesting gate on it yet (IMPLEMENTATION_PLAN.md section 10.2's backtest-validation requirement is not implemented — tracked for the dedicated forecasting phase, section 12 phase 9).

build_forecaster()

The one factory function that turns a `ForecastConfig` plus the ambient `window/metrics` values into a live `Forecaster` instance. Nothing else in the codebase should construct a `QuantileForecaster/SeasonalNaiveForecaster/HoltWintersForecaster` directly once a caller exists that needs one from config — route it through here so the model-name-to-class mapping stays in one place.

The Prometheus client and verify-metrics

What does this page answer? How does `metrics.py` talk to Prometheus without ever touching the network in a test, and what does each of the six `verify-metrics` checks actually query? Describes `proxmox_storage_drs/metrics.py`.

`_SessionLike / _ResponseLike: why not requests.Session directly`

`PrometheusClient.__init__` accepts `session: _SessionLike | None`, a Protocol defined in this module with only the one `get(...)` method the client actually calls — not `requests.Session` itself. `.agents/testing.md` forbids any test talking to a real Prometheus; a structural protocol lets a test hand in a small dataclass with a matching `get()` instead of mocking or subclassing the real (network-capable) `session`. `PrometheusClient()` with no `session` argument still constructs a real `requests.Session()`, which satisfies the protocol structurally — nothing special is needed to make the real thing fit.

`_get()`: one request path, three endpoint shapes

Every one of `instant_query`, `range_query` and `label_values` goes through the private `_get()`, which does the HTTP call, checks the status code, parses JSON, checks Prometheus’s own status: `"success"` field, and returns `payload["data"]`. That field’s *shape* genuinely differs by endpoint — a dict with a `result` list for `query/query_range`, a bare list for `label/<name>/values` — which is why `_get()` is typed `Any` rather than picking one shape; each public method knows which shape it asked for and narrows accordingly. Every failure mode (a transport error, a non-200 status, unparseable JSON, an unsuccessful Prometheus response) raises `MetricsError` from exactly this one place, so a caller catching `MetricsError` around any of the three public methods sees every failure mode uniformly.

PromQL construction

`build_rate_promql()` and `build_quantile_over_time_promql()` are pure string-building functions, deliberately factored out of any query-issuing code so they are unit-testable without a fake session at all — see `IMPLEMENTATION_PLAN.md` section 3.4 for the exact expressions they implement. `raw_metric_name()` is the one place that maps a `RAW_METRIC_FIELDS` entry (`"read_ops"`, ...) to the configured metric name on a `MetricsConfig`, via `getattr` — this is what lets `verify_metrics()` iterate “every configured raw metric” without hand-listing the six field names a second time anywhere in this module.

`verify_metrics()`: six checks, six functions

Each `_check_*` function implements exactly one `IMPLEMENTATION_PLAN.md` section 3.3 numbered check and returns `Findings` (plus, where relevant, the data the report carries forward):

Function	Section 3.3 step	Returns
<code>_check_metric_names_exist</code>	1	findings only
<code>_check_sample_series</code>	2 + 3 (labels present)	findings, { <code>metric_name</code> : <code>sample_labels</code> }
<code>_check_device_label_collision</code>	4	one <code>Finding</code> or <code>None</code> (pure, no I/O)
<code>_check_coverage</code>	5	findings, { <code>DiskKey</code> : <code>coverage_fraction</code> }
<code>_check_observed_spacing</code>	6	findings, <code>observed_spacing_seconds</code>

`_check_coverage` and `_check_observed_spacing` both use `metrics.read_ops` as the one representative metric rather than probing all six: a coverage or spacing gap is a property of the underlying Telegraf scrape, not of which of the six raw quantities is read, so probing all six would be six times the Prometheus load for no additional information.

`VerifyMetricsReport.ok` is `True` iff no `Finding` has `level == "error"` — `cli.py`’s `verify-metrics` handler uses exactly this property to decide the process exit code, so there is one definition of “the verification passed.”

What `cli.py` does with the report

`_handle_verify_metrics` in `cli.py` constructs a `PrometheusClient` from `resolved.config.prometheus`, calls `verify_metrics()`, and renders either the human form (`[level]` message lines) or, under `--json`, a JSON-serializable dict — `DiskKey` keys are turned into `"vmid:device"` strings there, since JSON object keys must be strings and this module keeps no opinion about output formatting.

Command dispatch, the mode-override rule, and why logs go to stderr

What does this page answer? How does `cli.py` turn `argv` into a dispatched command, what exactly does overriding `--mode log`, and why does structured logging go to `stderr` instead of the `stdout` the

plan originally specified? Describes proxmox_storage_drs/cli.py and proxmox_storage_drs/logging_setup.py.

One parser, global options first

build_parser() defines every global option (-c/--config, --group, --mode, --json, -v/--quiet, --version, --manual) on the **top-level** ArgumentParser, not on the subparsers, which is what makes argparse itself enforce IMPLEMENTATION_PLAN.md section 11.3’s “global options before the subcommand” for free — there is no hand-written ordering check anywhere. --help’s defaults come from the real default= values passed to add_argument, combined with argparse.ArgumentDefaultsHelpFormatter; there is deliberately no second, hand-maintained list of options and defaults anywhere in this module, the manpage, or the manual (AGENTS.md section 8.5).

main() handles --version and --manual/help **before** requiring or loading any configuration — printing the version or the manual page must never depend on /etc/pve/drs.yaml existing. Only once a real subcommand is present does it call config.load_config().

Command handlers: a dict, not a chain of ifs

_COMMAND_HANDLERS: dict[str, CommandHandler] maps each subcommand name to a function of (ResolvedConfig, argparse.Namespace, effective_mode) -> int. Every subcommand not yet implemented (plan, apply, show-load, explain, verify-storages — see [../manual/30-safety-and-status.md](#) for which ones that currently is) gets a handler from _make_not_yet_implemented_handler(name), a closure factory rather than a lambda with a default-argument trick, because mypy’s strict mode cannot infer a bare lambda’s parameter types cleanly here — see the comment at that function if you are tempted to simplify it back to a one-liner. _COMMAND_HANDLERS["verify-metrics"] is overwritten with the real _handle_verify_metrics once metrics.py exists to back it. Adding a new implemented command is exactly this: write the handler, assign it into the dict, done — main()’s dispatch does not change.

The --mode escalation rule

apply_mode_override(configured_mode, override) is the entire implementation of section 11.3’s rule: moving *down* the ordering dry-run < confirm < auto (toward more caution) logs at INFO; moving *up* it (toward less caution — removing a barrier the operator’s own config set) logs at WARNING, naming both values. This is deliberately a small pure-ish function (its only side effect is the log call) rather than inlined into main(), so it is unit-testable against a caplog fixture without constructing a full CLI invocation.

Why logs go to stderr, not stdout

IMPLEMENTATION_PLAN.md section 2.1 originally specified stdout for structured JSON logs. This was amended in the same commit that added logging_setup.py: --json (section 9.5) emits the plan report on **stdout**, and interleaving log lines with that report on the same stream would make it unparseable by anything downstream. Logging instead goes to **stderr**; a systemd service unit captures both streams into the same journal regardless, so nothing about “captured by journald” is lost. This is the one place in the codebase where the implementation and IMPLEMENTATION_PLAN.md disagree with the *original* plan text on purpose — AGENTS.md section 7 rule 2 is why the plan text itself was corrected in the same commit rather than left to silently diverge from the code.

configure_logging() is called exactly once, from main(), after the --version/--manual early exits (which must not touch logging configuration at all) and before any config-dependent code runs, so that a config-loading failure is itself logged consistently with everything after it.

JsonFormatter: what ends up in one log line

Every field on a logging.LogRecord that is *not* one of the standard attributes (computed once, at import time, into _STANDARD_RECORD_ATTRS) is folded into the JSON payload — this is what lets any

module call `logger.info(..., extra={"event": "gate_decision", "threshold": 0.2})` and have `event/threshold` appear as top-level JSON keys with no formatter changes required. `sort_keys=True` keeps output byte-stable for anything that diffs log lines in a test or a log-aggregation pipeline.

--manual: preferring man(1), falling back only when necessary

`show_manual()` tries `man pve-storage-drs` first, via `subprocess.run` rather than `os.execvp`: `man(1)` itself detects a non-tty stdout and disables its pager automatically, which is what satisfies “never answer with a URL alone” (AGENTS.md section 8.5) whether the invocation is interactive or piped — there is no need for this module to duplicate that tty detection. `_fallback_manual_text()` is reached only when `man(1)` itself is missing or has no entry (an uninstalled source checkout, or a minimal system with no `man-db`): it walks up from `cli.py`’s own file location looking for `man/pve-storage-drs.1.md` in a source tree, and only if that also fails prints a short message naming the real installed-package path and the in-tree file — a local, actionable path, never a bare URL.